

BLOC 4

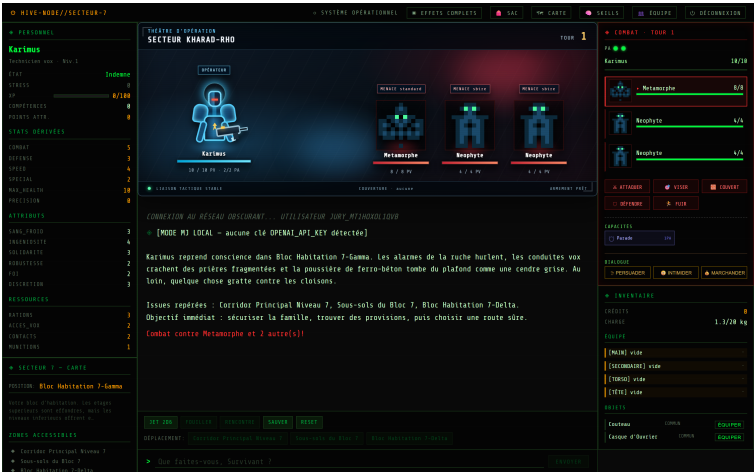
Maintenir l’application logicielle en condition operationnelle

Titre professionnel **Expert en developpement logiciel** — RNCP 39583

PROJET SUPPORT

RPG 40K — Survivant de Ruche

Jeu de role narratif solo. Architecture FastAPI + React, narration deleguee a un modele de langage, deploiement conteneurise sur VPS.



AUTEUR

NEHJMEHDINE REBIAI

Version documentee : 1.3.0 • 19 aout 2026

Avant-propos

Ce dossier presente le dispositif de maintien en condition operationnelle du projet **RPG 40K** — **Survivant de Ruche** : gestion des dependances, supervision et alerte, collecte et traitement des anomalies, deploiement des correctifs, axes d'amelioration, journal des versions et collaboration avec le support.

Parti pris. Aucun processus decrit ici n'est depourvu d'outillage dans le depot. Lorsqu'une competence exigeait un dispositif absent, celui-ci a ete *developpe*, execute et verifie avant d'etre documente. Les trois anomalies presentees sont des defauts **reels**, decouverts au cours de ce travail, reproduits et couverts par des tests qui echouent sur le code anterieur. Les releves reproduits sont des sorties reelles, y compris lorsqu'elles montrent un echec.

Chiffres clés

Tests backend (avant → apres)	82 → 135
Tests frontend	30
Anomalies reelles corrigees	3
Sondes de supervision	19
Regles d'alerte	extbf18
Recommandations argumentees	8

Limites assumees

1. Aucune base d'utilisateurs constituee : le canal de retour a ete mis en place, mais aucun retour reel n'a pu etre analyse (§7).
2. Environnement Docker indisponible : la pile de supervision est configuree et validee syntaxiquement, non executee.
3. Support exerce par une personne seule : les roles decrits sont des *fonctions*, non des equipes distinctes (§9).

Table des matières

1 Le logiciel supervise	2
2 Processus de mise a jour des dependances	2
3 Systeme de supervision et d'alerte	5
4 Collecte et consignation des anomalies	12
5 Fiche de consignation — ANO-2026-001	13
6 Traitement de l'anomalie par la chaine CI/CD	14
7 Recommandations argumentees d'amelioration	17
8 Journal des versions	19
9 Probleme resolu avec le support client	20
10 Conclusion	22
11 Annexe A — Artefacts ajoutes au depot	23
12 Annexe B — Inventaire des tests	23

1 Le logiciel supervise

Le logiciel n'est ni un site vitrine ni une API transactionnelle : c'est un **jeu de role narratif mono-joueur**, expose en REST + SSE, dont la narration est deleguee a un **fournisseur LLM externe** et dont l'etat de partie vit **en memoire** du processus avant serialisation sur disque. Quatre caracteristiques commandent tout le dispositif decrit ensuite.

Caracteristique	Consequence
Degradation silencieuse prevue par conception : en cas de panne du LLM, l'application bascule sur un narrateur local et repond toujours 200 OK	Une supervision fondee sur les codes HTTP declarerait le service sain alors que la valeur percue s'est effondree. Une sonde du mode de narration est indispensable.
Reponses en streaming SSE de plusieurs dizaines de secondes	Ces routes ne peuvent pas etre jugees sur les seuils de latence des routes de jeu ; elles sont exclues des alertes de lenteur.
Sessions en memoire , sans expiration	La consommation croit avec le nombre de joueurs depuis le demarrage : jauge de capacite.
Fournisseur facture a l'usage , historique jamais tronque	Le cout et la latence par appel croissent au fil de la partie : sonde de taille de contexte.

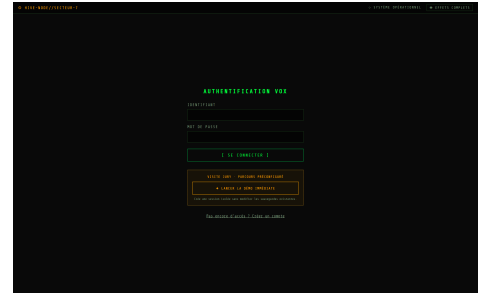


Figure 1 – Authentication JWT.

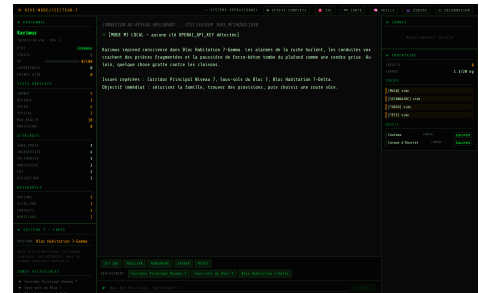


Figure 2 – Scene d'ouverture diffusee en SSE, en mode narrateur local.

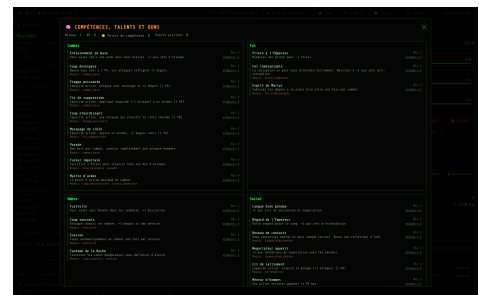


Figure 3 – Panneau de progression : arbre de competences et points d'attribut.

2 Processus de mise a jour des dependances

Compétence C4.1.1 — Gerer les mises a jour des dependances et des bibliotheques tiers, en surveillant regulierement les nouvelles versions, en evaluant les impacts des mises a jour, et en les integrant de maniere securisee.

2.1 Perimetre et politique

Le perimetre couvre quatre ensembles : **backend Python** (requirements.txt : FastAPI, Uvicorn, sse-starlette, PyJWT, bcrypt, openai, PyYAML, pytest) ; **frontend npm** (package.json + package-lock.json : React 19, Three.js en execution ; Vite, Vitest, Playwright en developpement) ; **images Docker** (python:3.13-slim, node:22-alpine, nginx:1.27-alpine) ; **chaîne CI/CD** (actions GitHub).

Cette separation est structurante : une regression sur une dependance de developpement n'affecte que le build, une regression sur une dependance d'execution atteint le service rendu.

Type	Exemple	Mode	Justification
Patch x.y.Z	bcrypt 5.0.0 -> 5.0.1	Automatique	Retro-compatibilite acquise; la CI valide la non-regression.
Mineure x.Y.z	fastapi 0.141 -> 0.142	Semi-auto	Auto sur develop, validation humaine avant production.
Majeure X.y.z	React 19 -> 20	Manuelle	Rupture probable : branche dediee, guide de migration, tests e2e.
Image Docker	python:3.13-slim	Automatique	Reconstruite a chaque deploiement (correctifs systeme).

2.2 Incident ayant motive le bornage

Defaut identifie — des montees majeures non decidees

requirements.txt ne declarait que des planchers (>=). Chaque installation propre et chaque execution de CI embarquaient donc des montees **majeures** que personne n'avait decidees ni evaluees.

Dependance	Plancher declare	Installee	Ecart
openai	>=1.51.0	3.3.0	2 majeures
rich	>=13.8.1	15.0.0	2 majeures
bcrypt	>=4.2.0	5.0.0	1 majeure
pytest	>=8.3.2	9.1.1	1 majeure

Le cas sensible est openai, dont depend tout le moteur de narration. **Evaluation d'impact** : la surface reellement appelee (AsyncOpenAI, chat.completions.create avec messages et stream) reste disponible en 3.3.0 et les 82 tests passaient. La version 3.3.0 devient donc le **nouveau plancher valide**, plafonne a <4.0.0 : la derive est convertie en decision tracee.

2.3 Outillage mis en place

1. Bornage — chaque dependance porte un plancher (version validee) et un plafond (interdit la majeure). La regle « majeure = manuelle » n'est plus une consigne : elle est appliquee par pip.

```
fastapi>=0.141.1,<1.0.0    openai>=3.3.0,<4.0.0    bcrypt>=5.0.0,<6.0.0
```

2. Detection — .github/dependabot.yml inspecte chaque lundi les 4 ecosystemes. Mineures et correctifs **groupes**, majeures en **PR isolee** pour forcer une revue individuelle.

3. Veille — scripts/check_updates.py compare les versions majeures et classe les ecarts ; code de sortie 1 si une majeure est en attente. Integre au job security-audit.

```
MONTEES MAJEURES - traitement MANUEL (3)
  npm @testing-library/jest-dom 6.9.1 -> 7.0.1    npm jsdom 29.1.1 -> 30.0.1
  npm typescript                  6.0.3 -> 7.0.2
=> Ouvrir un ticket par montee, evaluer l'impact, brancher sur feature/.
MINEURES ET CORRECTIFS - traitement AUTOMATIQUE (10) => Lot mensuel, valide par la CI.
```

2.4 La politique observee en fonctionnement

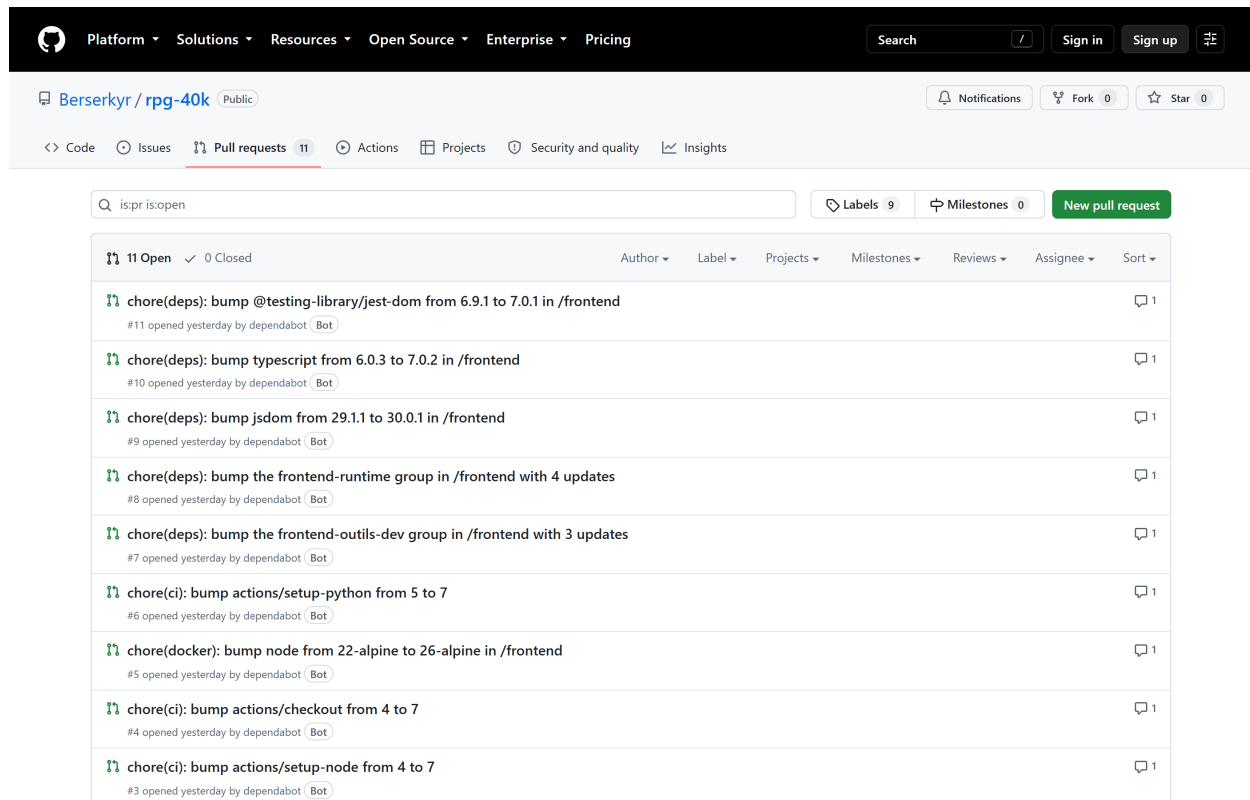


Figure 4 — Pull requests reellement ouvertes par Dependabot sur le depot. Trois comportements prevus par la configuration se lisent directement : les montees **groupees** (*frontend-runtime*, 4 mises a jour ; *frontend-outils-dev*, 3 mises a jour), les montees **majeures isolees** chacune dans sa propre PR (*jest-dom* 6 → 7, *typescript* 6 → 7, *jsdom* 29 → 30), et la couverture effective des **quatre ecosystemes** — npm, GitHub Actions, Docker (*node 22-alpine* → *26-alpine*) et pip.

Cette copie d’ecran n’illustre pas la politique : elle la **demonstre**. Le groupement et l’isolement des majeures ne sont pas des intentions declarees dans un fichier de configuration, ce sont des comportements observables sur le depot. On note au passage que *node* passerait de la version 22 a la version 26 : c’est exactement le type de saut qui, sans la regle d’isolement, serait passe inaperçu au milieu d’un lot.

2.5 Frequence et evaluation d’impact

Activite	Frequence	Outillage
Audit de securite	A chaque push / PR	<code>pip-audit</code> , <code>npm audit</code>
Rapport de veille	A chaque push / PR	<code>scripts/check_updates.py</code>
Detection des nouvelles versions	Hebdomadaire (lundi 06h00)	<code>dependabot.yml</code> , 4 ecosystemes
Lot de mises a jour mineures	Mensuel	PR groupees, fusion apres CI verte
CVE elevee ou critique	Sous 48 h	Hors lot, branche <code>fix/</code>
Montees majeures	Trimestrielle	PR isolee, apres evaluation d’impact

Trois questions sont tranchees avant integration : la dependance est-elle **embarquee en production** ? quelle est sa **surface d’appel** ? le correctif est-il **disponible sans rupture** ?

Application reelle : l’audit npm a remonte 4 vulnerabilites « high » (*vite*, *postcss*, *nanoid*, *undici*). Toutes sont des dependances de **developpement**, absentes du bundle servi par Nginx, et leur vecteur suppose un acces au serveur de developpement. **Decision** : lot mensuel via `npm audit`

fix, et non correctif d'urgence sous 48 h.

Couverture des criteres

Critere	Traitement	Preuve
Frequence des mises a jour	Audit et veille a chaque push, detection hebdomadaire, lot mensuel, CVE sous 48 h, majeures trimestrielles	<code>dependabot.yml</code> , <code>job security-audit</code>
Perimetre logiciel	Backend Python, frontend npm, images Docker, chaine CI/CD	4 entrees <code>updates:</code>
Type automatique / manuel	Patch auto, mineure semi-auto, majeure manuelle	Bornes <code><N.0.0</code> , <code>groups</code>

3 Systeme de supervision et d'alerte

Compétence C4.1.2 — Concevoir un systeme de supervision et d'alerte en determinant le perimetre, en identifiant les indicateurs pertinents, en mettant en place des sondes et en configurant la modalite des signalements afin de garantir une disponibilite permanente.

Etat initial — une sonde qui n'interroge rien

Le projet ne disposait d'aucune supervision. `/api/health` retournait un dictionnaire fige affichant le *chemin* de la base sans jamais l'interroger : une base corrompue, un volume non monte ou un disque plein laissaient la sonde repondre `"status": "ok"` alors que toute partie etait impossible.

3.1 Perimetre : ce que Prometheus et Grafana peuvent superviser

Prometheus et Grafana supervisent le **serveur**, l'**API** et l'**infrastructure**. Ils ne mesurent pas le rendu du jeu dans le navigateur : ce qui se passe apres la reponse HTTP — temps de peinture, fluidite de l'animation, erreurs JavaScript cote client — releve d'un outillage different (instrumentation navigateur de type RUM), hors du perimetre de ce dossier. La distinction est importante : elle delimite ce que la supervision peut affirmer et ce qu'elle ne peut pas.

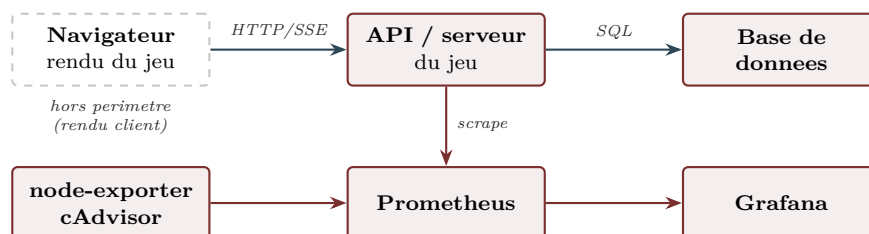


Figure 5 — Le trait plein suit le parcours d'une requete joueur ; les fleches sombres, la chaine de supervision. Le navigateur est le seul maillon non instrumente.

Domaine	Ce qui est supervise	Par quoi
Serveur	CPU, memoire, espace disque, reseau de l'hote ; consommation et redemarrages par conteneur	<code>node-exporter</code> , <code>cAdvisor</code>
Backend / API	Temps de reponse (centiles), nombre de requetes par route, codes HTTP, exceptions non gerees	Instrumentation applicative
Jeu	Sessions actives, flux SSE ouverts, combats engages par faction, mode de production de la narration	Sondes metier
Base de donnees	Disponibilite verifiee par une <i>requete réelle</i> , duree d'écriture des sauvegardes, echecs d'écriture, volumetrie	Sonde d'aptitude, <code>benchmark.py</code>
Erreurs	Taux de 4xx et 5xx, exceptions, dependance indisponible, service démarre mais inapte	Middleware et sondes de dependance

Une nuance sur la base de donnees. Les indicateurs habituels — nombre de connexions, saturation du pool, requetes lentes — supposent un SGBD *serveur*. Le projet utilise **SQLite**, moteur embarque dans le processus : il n’y a ni connexion reseau ni pool a surveiller. Ce qui reste pertinent est donc l’**accessibilite** du fichier (verifiee par une requete reelle a chaque collecte), la **duree d’ecriture** des sauvegardes et la **volumetrie**. Transposer mecaniquement un tableau de bord MySQL ou PostgreSQL produirait des panneaux vides. Une migration vers PostgreSQL justifierait en revanche l’ajout de `postgres_exporter`.

3.2 Sondes de disponibilite : vivacite contre aptitude

Sonde	Route	Finalite	Effet d’un echec
Vivacite	GET /api/health	Atteste que le processus repond	Redemarrage du conteneur
Aptitude	GET /api/health/ready	Verifie <i>reellement</i> les dependances	503 et retrait du trafic, sans redemarrage

Cette separation evite un piege d’exploitation : si la sonde de redemarrage verifiait la base, une indisponibilite disque declencherait une **boucle de redemarrage** aggravant la panne. La sonde d’aptitude effectue quatre controles **actifs** : requete reelle `SELECT count(*) FROM users`, presence et taille non nulle de `character_sheet.yaml` et `prompt_survivant.md`, ecriture puis suppression d’un fichier temoin dans le repertoire de sauvegarde.

Pretty-print ☐

```
{"status":"ready","checks":{"database":{"ok":true},"character_sheet":{"ok":true},"prompt_file":{"ok":true},"save_dir":{"ok":true},"llm_configured":false}}
```

Figure 6 – Sonde d’aptitude en fonctionnement : chaque dependance est controlee individuellement.

3.3 Sondes applicatives

La sonde structurante est `rpg40k_gm_generations_total{mode}`, qui distingue `openai`, `local_fallback` et `local_no_key` : elle rend **observable** la bascule silencieuse vers le narrateur local, seule facon de voir une panne du fournisseur derriere un 200 OK.

Sonde	Type	Finalite
<code>rpg40k_gm_generations_total{mode}</code>	Compteur	Mode de production reel de la narration
<code>rpg40k_gm_generation_duration_seconds</code>	Histogramme	Temps de generation percu
<code>rpg40k_gm_context_messages</code>	Histogramme	Croissance de l’historique envoye au LLM
<code>rpg40k_http_requests_total{...}</code>	Compteur	Trafic et taux d’erreur par famille de code
<code>rpg40k_http_request_duration_seconds</code>	Histogramme	Centiles de latence
<code>rpg40k_auth_attempts_total{action,result}</code>	Compteur	Detection de tentatives repetees
<code>rpg40k_active_sessions / rpg40k_sse_streams_active</code>	Jauges	Capacite et charge reelle
<code>rpg40k_state_markers_rejected_total</code>	Compteur	Marqueurs d’etat ignores (§5)
<code>rpg40k_ready / rpg40k_dependency_up</code>	Jauges	Etat des dependances internes

Maitrise de la cardinalite : les chemins porteurs d’identifiants sont ramenes a leur gabarit (`/api/animations/tir_bolter` devient `/api/animations/{skill_id}`). Sans cette normalisation, chaque identifiant creerait une serie temporelle distincte. Un test verifie que cent identifiants ne produisent qu’un seul libelle.

```
# HELP rpg40k_build_info Informations de version du service
# TYPE rpg40k_build_info gauge
rpg40k_build_info{service="survivant-de-ruche-api",version="1.3.0"} 1.0
# HELP rpg40k_http_requests_total Nombre total de requêtes HTTP traitées.
# TYPE rpg40k_http_requests_total counter
rpg40k_http_requests_total{endpoint="/api/health",method="GET",status_class="2xx"} 54.0
rpg40k_http_requests_total{endpoint="/api/auth/register",method="OPTIONS",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/auth/register",method="POST",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/state",method="OPTIONS",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/state",method="GET",status_class="2xx"} 2.0
rpg40k_http_requests_total{endpoint="/api/start",method="OPTIONS",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/start",method="POST",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/combat/start",method="OPTIONS",status_class="2xx"} 1.0
rpg40k_http_requests_total{endpoint="/api/combat/start",method="POST",status_class="2xx"} 1.0
# HELP rpg40k_http_requests_created Nombre total de requêtes HTTP traitées.
# TYPE rpg40k_http_requests_created gauge
rpg40k_http_requests_created{endpoint="/api/health",method="GET",status_class="2xx"} 1.7872282336095567e+09
rpg40k_http_requests_created{endpoint="/api/auth/register",method="OPTIONS",status_class="2xx"} 1.7872284812236252e+09
rpg40k_http_requests_created{endpoint="/api/auth/register",method="POST",status_class="2xx"} 1.787228481461567e+09
rpg40k_http_requests_created{endpoint="/api/state",method="OPTIONS",status_class="2xx"} 1.7872284815666966e+09
rpg40k_http_requests_created{endpoint="/api/state",method="GET",status_class="2xx"} 1.787228481579498e+09
rpg40k_http_requests_created{endpoint="/api/start",method="OPTIONS",status_class="2xx"} 1.7872284817698753e+09
rpg40k_http_requests_created{endpoint="/api/start",method="POST",status_class="2xx"} 1.787228481777095e+09
rpg40k_http_requests_created{endpoint="/api/combat/start",method="OPTIONS",status_class="2xx"} 1.7872284845706286e+09
rpg40k_http_requests_created{endpoint="/api/combat/start",method="POST",status_class="2xx"} 1.7872284846086125e+09
# HELP rpg40k_http_request_duration_seconds Durée de traitement des requêtes HTTP.
# TYPE rpg40k_http_request_duration_seconds histogram
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.01",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.025",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.05",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.1",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.25",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="0.5",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="1.0",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="2.5",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="5.0",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="10.0",method="GET"} 54.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/health",le="+Inf",method="GET"} 54.0
rpg40k_http_request_duration_seconds_count{endpoint="/api/health",method="GET"} 54.0
rpg40k_http_request_duration_seconds_sum{endpoint="/api/health",method="GET"} 0.02842109987977892
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.01",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.025",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.05",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.1",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.25",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="0.5",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="1.0",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="2.5",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="5.0",method="OPTIONS"} 1.0
rpg40k_http_request_duration_seconds_bucket{endpoint="/api/auth/register",le="10.0",method="OPTIONS"} 1.0
```

Figure 7 – Point de collecte `/api/metrics` apres un parcours reel : version du service, trafic par route et histogrammes de latence.

3.4 Criteres de qualite et modalite des signalements

Indicateur	Objectif	Seuil d’alerte	Justification
Disponibilite API	≥ 99 %/mois	Injoignable > 1 min	Jeu solo : une coupure breve est toleree
Latence p95 routes de jeu	< 250 ms	> 1 s pendant 10 min	Calculs en memoire ; mesure : 1,0 ms sur <code>/api/roll</code>
Latence <code>/api/auth/login</code>	< 500 ms	—	Mesure : ~230 ms dont 219 ms de bcrypt . Lenteur voulue, a ne pas optimiser
Taux d’erreur 5xx	< 1 %	> 5 % pendant 5 min	Une erreur interrompt une partie
Narration servie par le LLM	100 %	≥ 1 repli en 10 min	Tout <code>local_fallback</code> traduit une panne
Taille du contexte LLM	< 100 messages	p95 > 200	Impact direct sur cout et latence
Echecs d’authentification	—	> 10/min pendant 5 min	Signature d’une recherche exhaustive

Les paliers de l’histogramme (0.01 a 10 s) sont resserres sous la seconde, la ou se situe la distribution réelle ; des paliers generiques auraient rendu le p95 inexploitable.

3.5 Catalogue des 18 regles d’alerte

Groupe	Regle	Severite	Condition et intention
disponibilite	ServiceIndisponible	critique	up == 0 pendant 1 min : le collecteur ne joint plus le backend
	ServiceNonPret	critique	rpg40k_ready == 0 : le processus repond mais une dependance manque
	DependanceIndisponible	critique	Identifie <i>laquelle</i> des quatre dependances est tombee
	SondeExterneEnEchec	critique	Sonde boite noire : couvre aussi une panne du reverse proxy
ressources serveur	EspaceDisqueFaible	majeure	< 15 % d’espace libre : les sauvegardes echoueront avant tout signal applicatif
	EspaceDisqueCritique	critique	< 5 % : perte de donnees imminente
	MemoireHoteSaturee	majeure	> 90 % pendant 10 min : risque d’arret par le noyau
	ChargeProcesseurElevee	mineure	> 85 % pendant 15 min : sur un service a faible trafic, signale plutot une boucle
	ConteneurRedemarrages	majeure	Redemarrage inattendu d’un conteneur
qualite de service	TauxErreurServeurEleve	majeure	> 5 % de 5xx sur 5 min
	LatenceApiDegradee	majeure	p95 > 1 s sur 10 min, hors routes de streaming
	ExceptionsNonGerees	majeure	Toute exception remontee jusqu’au middleware
dependance LLM	ModeDegradeNarration	majeure	Regle structurante : tout repli sur le narrateur local
	LatenceLlmElevee	mineure	p95 de generation > 20 s
	ContexteConversationExcessif	mineure	p95 > 200 messages : derive de cout et de latence
securite	EchecsAuthentificationRepetes	majeure	> 10 echecs/min : recherche exhaustive probable
capacite	SessionsMemoireCroissantes	mineure	> 200 sessions residentes pendant 15 min
	EchecSauvegarde	critique	Toute ecriture de sauvegarde en echec : perte de progression

Les fenetres **for** : sont volontairement superieures a l’intervalle de collecte, afin qu’un pic isole ne declenche pas d’alerte.

3.6 Modalite des signalements

Le routage Alertmanager traduit la severite en canal *et* en cadence.

Severite	Destinataire	Groupe	Rappel	Intention
Critique	Astreinte, notification immediate	10 s	1 h	Service indisponible ou donnees menacees
Majeure	Equipe, heures ouvrees	1 min	4 h	Degradation perceptible par le joueur
Mineure	Consignation seule	30 s	24 h	Derive a surveiller, revue hebdomadaire

Deux mecanismes limitent le bruit, condition d’une supervision *reellement* exploitee. Le **groupe-ment** (group_by: [alertname, severite]) fait qu’une panne declenchant dix regles produit une notification par famille et non dix. L’**inhibition** supprime les alertes de latence et de taux d’erreur lorsque **ServiceIndisponible** est actif, puisqu’elles en decoulent mecaniquement. Les URL de destination ne sont jamais versionnees : elles proviennent des variables **ALERT_WEBHOOK_CRITIQUE** et **ALERT_WEBHOOK_EQUIPE**.

3.7 Architecture de la chaine

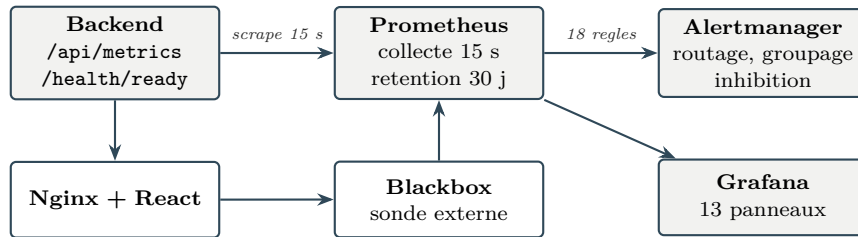


Figure 8 – Collecte applicative, sonde externe « boîte noire », évaluation des règles et visualisation. Prometheus se supervise lui-même : sans cela, l’arrêt du collecteur produirait un silence indistinguable d’une absence d’incident.

La pile s’active en surcouches, ce qui laisse la pile applicative deployable seule. Prometheus, Alertmanager et Grafana n’exposent leurs ports que sur 127.0.0.1 : ils n’embarquent pas d’authentification suffisante pour une exposition publique, et l’accès distant passe par un tunnel SSH.

```
docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d
```

3.8 Validation de la pile en fonctionnement

La chaîne a été exécutée de bout en bout pour vérifier qu’elle ne se limite pas à des fichiers de configuration valides. Prometheus a collecté le backend réel, évalué les 18 règles, et Grafana a affiché le tableau de bord *provisionné depuis le dépôt*.

Première vérification, avec `promtool`, l’outil officiel de Prometheus — il contrôle la syntaxe *et* les expressions PromQL, ce qu’une simple validation YAML ne fait pas :

```

$ promtool check rules alert_rules.yml      $ promtool check config prometheus.yml
SUCCESS: 18 rules found                     SUCCESS: prometheus.yml is valid

$ curl -s localhost:9090/api/v1/targets
prometheus      http://127.0.0.1:9090/metrics    UP
rpg40k-backend  http://127.0.0.1:8000/api/metrics  UP

```

Une panne du fournisseur LLM a ensuite été simulée (cle API invalide) pendant que Prometheus collectait. L’alerte structurante est passée en état **firing** :

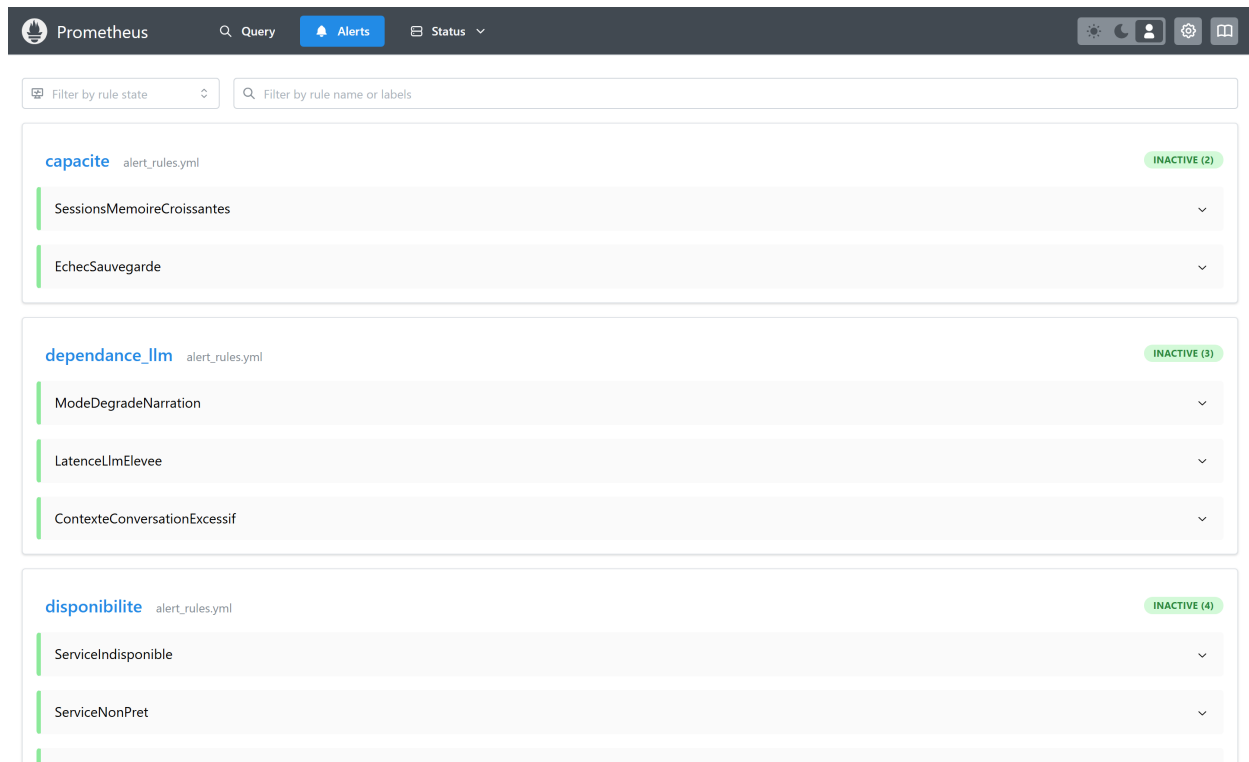


Figure 9 — Regles evalues par Prometheus. Le groupe `dependance_llm` affiche **FIRING (1)** : `ModeDegradeNarration` s'est declenchee parce que la narration etait servie par le narrateur local. L'application repondait pourtant 200 OK — c'est exactement la degradation qu'aucune supervision fondee sur les codes HTTP n'aurait vue.

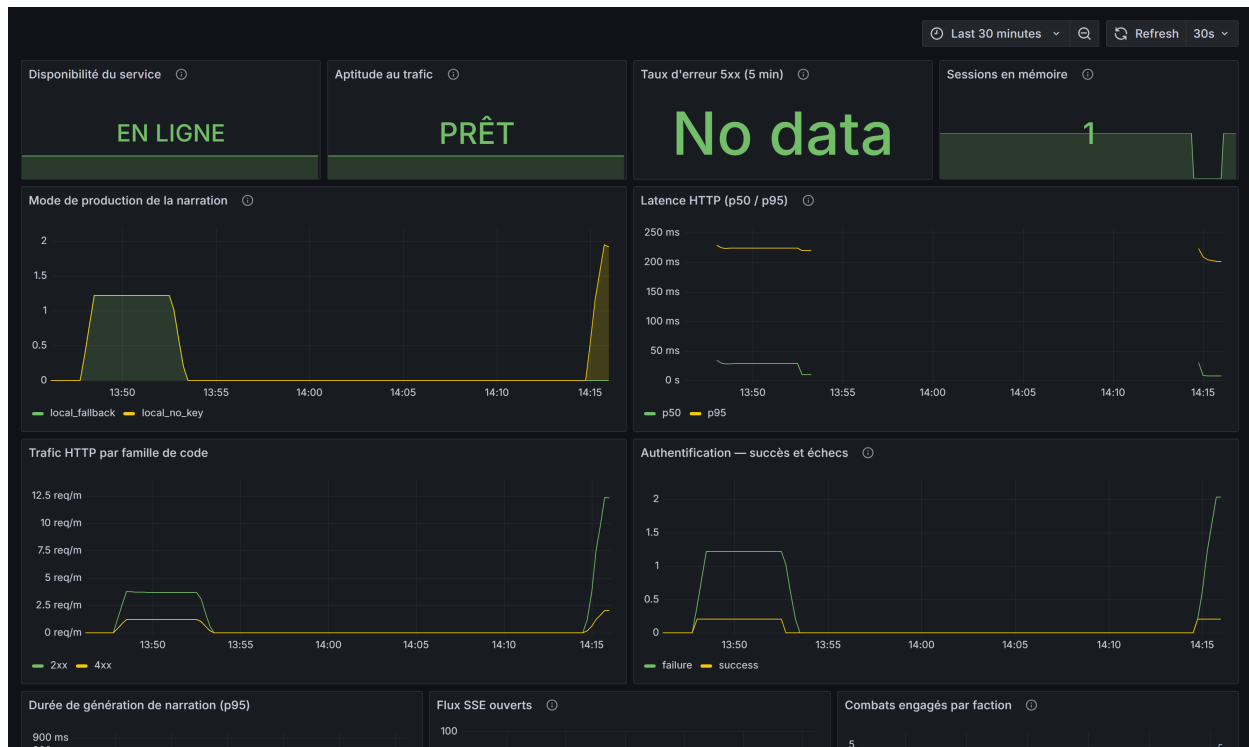


Figure 10 — Tableau de bord d'exploitation, alimente par des donnees reelles. Le panneau *Mode de production de la narration* distingue `local_fallback` de `local_no_key` ; le taux d'erreur 5xx affiche *No data*, ce qui est exact : aucune erreur serveur n'est survenue pendant la mesure.

3.9 Tableau de bord infrastructure et jeu

Un second tableau de bord regroupe les indicateurs d'exploitation courants : ressources de l'hôte, activité de jeu et santé de l'API sur une même vue.

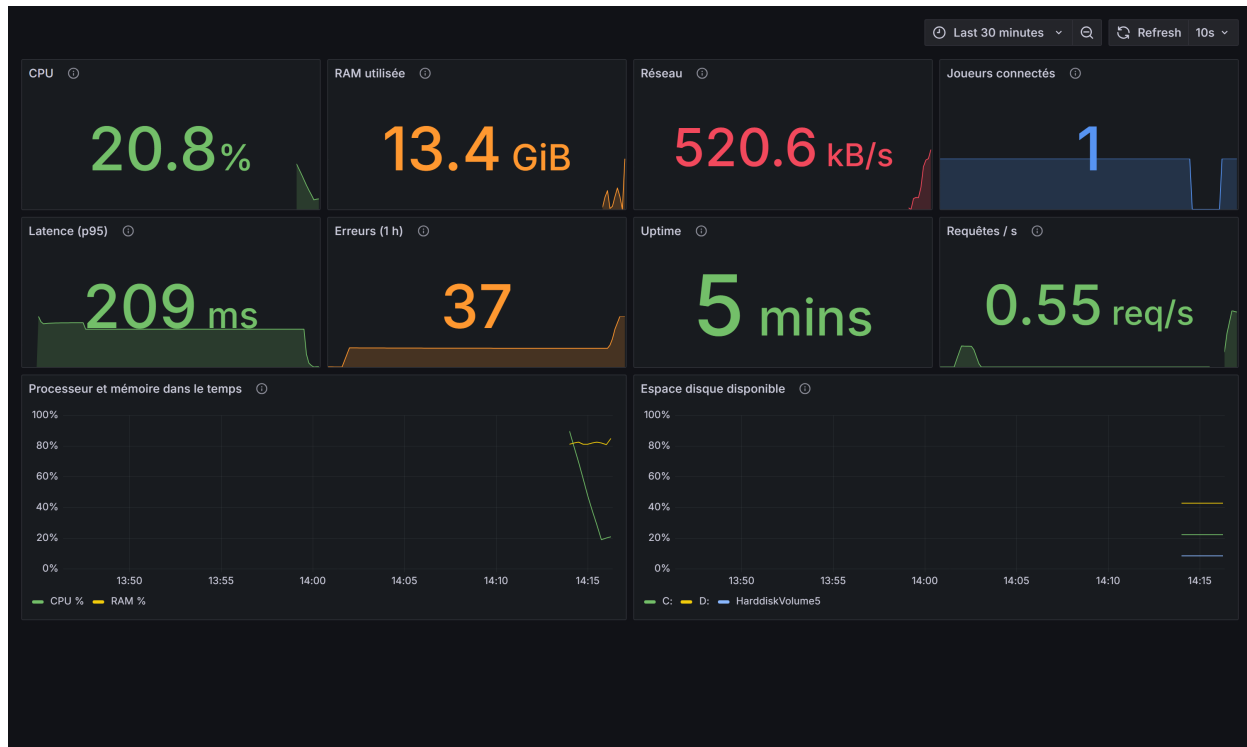


Figure 11 – Huit indicateurs alimentés par des mesures réelles : charge processeur, mémoire utilisée, débit réseau, joueurs connectés, latence p95, erreurs cumulées, uptime du processus et débit de requêtes. En bas, l'évolution processeur/mémoire et l'espace disque par volume, avec les seuils de 15 % et 5 % des règles EspaceDisque.

Indicateur	Source	Interet
CPU	<code>node_cpu_seconds_total</code>	Saturation de l'hôte
RAM utilisée	<code>node_memory_*</code>	Risque d'arrêt par le noyau
Réseau	<code>node_network_*_bytes_total</code>	Volume échange, détection d'anomalie
Joueurs connectés	<code>rpg40k_active_sessions</code>	Charge réelle et dérive mémoire
Latence p95	<code>rpg40k_http_request_duration_seconds</code>	Expérience perçue, hors streaming
Erreurs (1 h)	<code>rpg40k_http_requests_total</code>	Volume de 4xx et 5xx
Uptime	<code>rpg40k_process_start_time_seconds</code>	Détecte les redémarrages silencieux
Requetes / s	<code>rpg40k_http_requests_total</code>	Débit traité par l'API

La sonde d'uptime a été ajoutée pour ce tableau de bord : `prometheus_client` n'expose `process_start_time_seconds` sous Linux, ce qui aurait laissé la tuile vide sur une autre plateforme. Les panneaux de ressources sont écrits avec l'opérateur `or` de PromQL (`node_...` ou `windows_...`), de sorte que le même fichier fonctionne en production Linux comme en validation locale.

Portée de cette validation. Docker n'étant pas disponible sur le poste de travail, la pile a été exécutée à partir des **binaires autonomes** de Prometheus 3.1.0 et Grafana 11.5.1, avec les cibles adressées en `127.0.0.1` au lieu des noms de services Compose. Les **regles d'alerte et le tableau de bord sont ceux du dépôt, sans modification** ; seules les adresses des cibles diffèrent. `node-exporter`, `cAdvisor`, `blackbox-exporter` et `Alertmanager` restent donc configurés mais non exécutés.

3.10 Couverture des modes de defaillance

Mode de defaillance	Detection	Reaction
Processus arrete ou fige	<code>up == 0</code> (1 min)	Redemarrage automatique, alerte critique
Dependance en panne	<code>rpg40k_ready == 0</code> (2 min)	Retrait du trafic (503), pas de redemarrage
Panne du proxy ou du frontend	<code>probe_success == 0</code>	Alerte critique meme si le backend est sain
Service disponible mais degrade	<code>mode="local_fallback"</code>	Alerte majeure : degradation rendue visible

Le dernier cas est celui que la sonde initiale ne pouvait pas voir, et c'est le plus probable en exploitation : cle expiree, quota depasse, incident du fournisseur. **20 tests** couvrent le dispositif (82 → 102) et verifient que la sonde d'aptitude detecte *reellement* une base corrompue et un fichier de jeu absent.

Couverture des criteres

Critere	Traitement
Systeme adapte a la typologie	Quatre caracteristiques du logiciel traduites en choix de sondes et de seuils (§1)
Sondes et finalite explicitees	Disponibilite, applicatives, LLM, securite, capacite : type et finalite pour chacune
Criteres de qualite adaptes	Sept indicateurs, seuils justifies par des mesures reelles (1,0 ms ; 230 ms dont 219 de bcrypt)
Surveillance de la disponibilite	Quatre modes de defaillance couverts, dont la degradation silencieuse

4 Collecte et consignation des anomalies

Competence C4.2.1 — Consigner les anomalies detectees en elaborant un processus de collecte et consignation, en utilisant des outils de collecte et en y integrant toutes les informations pertinentes, afin de determiner le correctif a mettre en place.

4.1 Adaptation a la typologie du logiciel

1. Sorties non deterministes. La narration provenant d'un LLM, une anomalie declenchee par une formulation particuliere est **intermittente et non rejouable a l'identique**. La fiche impose donc *Frequence* et *Horodatage*, seuls moyens de retrouver le texte incrimine dans les journaux.

2. Degradation sans erreur visible. Une part des anomalies ne se manifeste **jamais** par un code d'erreur : elles ne sont detectables que par les sondes du §3. Un formulaire distinct existe pour ces incidents, qui n'ont pas de signalement utilisateur.

3. Sauvegarde par scene. Une anomalie en fin de traitement peut faire perdre une scene entiere : la gravite est evaluee a l'aune de la **perte de donnees**, non du seul blocage.

4.2 Sources, outils et cycle de vie

Source	Outil	Delai
Supervision	18 regles Prometheus / Alertmanager	Temps reel
Journaux applicatifs	Journalisation structuree	Temps reel
Integration continue	GitHub Actions (5 jobs)	A chaque push
Audit de dependances	<code>pip-audit</code> , <code>npm audit</code>	A chaque push
Signalement joueur	Formulaire GitHub Issues	Variable
Recette manuelle	Plan de recette	Avant production

Les quatre premieres sources sont **automatiques** : point decisif pour un logiciel a faible base d'utilisateurs, ou l'on ne peut compter sur le volume de signalements.

La collecte est centralisee dans GitHub Issues avec trois formulaires structures, la saisie libre

etant desactivee (`blank_issues_enabled: false`) : 01_anomalie.yml (10 champs obligatoires), 02_incident_supervision.yml (5), 03_retour_utilisateur.yml (4). Le formulaire contraint repond a un constat pratique : une description en prose omet presque toujours la version, l'environnement ou les etapes exactes, imposant un aller-retour avant analyse. Les dix champs obligatoires sont exactement ceux **sans lesquels le bogue n'est pas reproductible**.

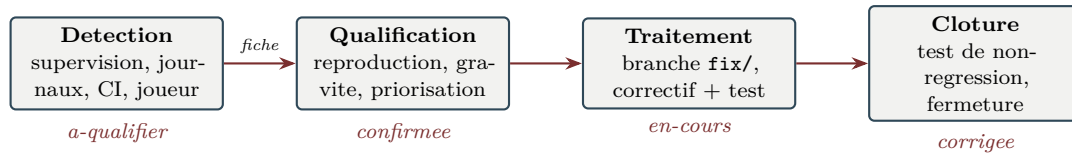


Figure 12 – Cycle de vie d’une anomalie ; les etats correspondent aux etiquettes GitHub et au kanban.

Grille de gravite — **Bloquante** (jeu inutilisable, immediat) ; **Majeure** (fonctionnalite inutilisable ou perte de donnees, 48 h) ; **Mineure** (gene sans perte, lot mensuel) ; **Cosmetique** (affichage, opportuniste). La perte de donnees est classee majeure *memes sans blocage* : dans un jeu a sauvegarde par scene, perdre une scene annule plusieurs minutes de jeu.

5 Fiche de consignation — ANO-2026-001

Competence C4.2.1 (livrable 2) — La presentation d’une fiche de consignation d’une anomalie rencontrée au cours du projet.

Identifiant	ANO-2026-001	Gravite	Majeure (perte de donnees)
Perimetre	Narration / maitre du jeu	Version	v1.0.0-rncp
Frequence	Intermittente, depend du texte du LLM	Etat	Corrigee en 1.3.0

5.1 Description et reproduction

A l’issue de certaines scenes, le joueur recoit integralement la narration puis l’interface reste bloquee sur l’indicateur d’attente, sans message d’erreur. En rechargeant, **la scene qui vient d’etre jouee a disparu**. Le phenomene est intermittent et independant de l’action du joueur, ce qui orientait a tort vers un probleme reseau.

Le declencheur etant une sortie du LLM, la reproduction fiable passe par la substitution du narrateur : demarrer une partie, puis faire produire une narration dont la valeur de ressource n’est **pas numerique**, et observer le flux SSE de POST /api/chat.

Karimus partage ses dernieres provisions. [ETAT: rations=aucune, stress=+1]

```

--- NOMINAL (rations=-1) ---
statut HTTP      : 200
evenements 'token' : 5
evenements 'done' : 1

--- DEFAILLANT (rations=aucune) ---
!! ValueError: invalid literal for
    int() with base 10: 'aucune'
-> aucun evenement 'done' emis
  
```

5.2 Analyse technique

Cause racine — dans `src/state.py`, `apply_updates_from_text()` convertit la valeur d’une ressource sans proteger la conversion. **Asymetrie revelatrice** : la branche voisine, qui traite **stress** et **corruption**, intercepte pourtant *exactement* le meme cas. Le default n’est donc pas un oubli de conception mais une **protection appliquee a une branche et omise sur l’autre**.

```

# branche resources : conversion nue -> ValueError
if value.startswith(('+', '-')): delta = int(value)
else:                             self.set_resource(key, int(value))

# branche tracks : le meme cas est deja intercepte
try:                         self.tracks[key] = int(value)
except ValueError:          self.tracks[key] = value
  
```

Mecanisme de l'impact — l'appel se situe dans le generateur SSE; l'ordre des instructions explique les deux symptomes simultanes.

```
546 session.messages.append({"role":"assistant","content": full_text})
548 changes = session.character.apply_updates_from_text(full_text) # exception ici
550 session.save() # jamais atteint
551 yield json.dumps({"type":"done", ...}) # jamais emis
```

L'exception survient **apres** l'envoi de la narration (le joueur a lu la scene), **avant** `session.save()` (progression perdue) et **avant** l'evenement `done` (client bloque). Le default avait echappe aux tests parce que la suite ne couvrait ce parseur qu'avec des valeurs numeriques valides.

5.3 Preconisations et verification

Option	Analyse	Retenue
A. Renforcer le prompt systeme	Ne supprime pas le risque : la sortie d'un LLM ne se garantit pas par une consigne	Non
B. Intercepter la <code>ValueError</code> dans le parseur	Traite la cause racine, aligne <code>resources</code> sur <code>tracks</code>	Oui
C. Protger l'appel dans le generateur SSE	Ne traite pas la cause, mais garantit qu'aucune future erreur ne coute une sauvegarde	Oui

B et C sont retenues ensemble, en **defense en profondeur** : B corrige le default identifie, C protege contre les defaults de meme famille non encore decouverts. Une quatrieme mesure rend le phenomene **mesurable** : les sondes `rpg40k_state_markers_rejected_total` et `rpg40k_state_parse_failures_total`. Un marqueur ignore reste en effet une anomalie mineure (la ressource n'est pas mise a jour alors que la narration le laisse entendre) : la sonde permet d'en mesurer la frequence residuelle.

18 tests de non-regression (dont 8 valeurs textuelles en parametre), portant la suite de 102 a **120**. Apres correctif, le meme parcours renvoie 200, 6 evenements `token`, 1 evenement `done`, et la sonde de marqueurs rejetees vaut 1.0.

Couverture des criteres

Critere	Traitement
Processus structure et adapte	Six sources dont quatre automatiques, trois formulaires, grille fondee sur la perte de donnees, cycle en quatre etats
Informations permettant de reproduire	Etapas, marqueur declencheur exact, commande de reproduction automatisee, releves avant/apres
Analyse et preconisations explicitees	Cause racine, asymetrie des branches, impact ligne a ligne ; trois options comparees, deux retenues, 18 tests

6 Traitement de l'anomalie par la chaine CI/CD

Competence C4.2.2 — Creer et deployer un correctif en respectant le processus d'integration et de deploiement continu afin de resoudre l'anomalie.

6.1 Chaîne mobilisée

#	Etape	Declencheur	Contrôle exercé
1	Branche de correction	Manuel	Isolation du correctif
2	Vérification locale	Manuel	Boucle courte avant push
3	Intégration continue	Push / pull request	5 jobs GitHub Actions
4	Revue	Pull request	Relecture du correctif et des tests
5	Fusion vers <code>main</code>	Manuel après CI verte	Seul un code teste atteint <code>main</code>
6	Déploiement	Automatique sur CI verte	Reconstruction et redéploiement Docker
7	Validation post-déploiement	Automatique	5 contrôles fonctionnels
8	Retour arrière	Automatique si échec	Restauration du commit précédent

6.2 Decoupage en commits

Le correctif est decoupe pour que **chaque commit reste independamment testable**, condition pour qu'une bisection (`git bisect`) demeure exploitable et qu'un retour arriere partiel soit possible.

```
9f98298 test(animations): rendre deterministe le test de fin d'animation (ANO-2026-003)
0098252 fix(sauvegarde): persister la fiche de personnage (ANO-2026-002)
ee546a3 feat(version): source unique de verite et journal des versions 1.3.0
5cafcbd feat(perf): outiller la mesure des indicateurs et la collecte des retours
8da4dca ci(deploy): valider le deploiement par test de fumee et rollback automatique
26df2a1 fix(narration): ne plus perdre la progression sur un marqueur d'etat invalide
b335510 feat(supervision): instrumenter l'application et outiller l'alerte
e9e52d5 chore(deps): borner les versions et automatiser la veille des dependances
```

L'ordre n'est pas arbitraire : le commit de supervision **precede** le correctif, car celui-ci s'appuie sur les sondes qu'il introduit. Le message suit la convention *Conventional Commits* déjà employée dans le depot et référence explicitement l'anomalie, ce qui relie le commit à sa fiche de consignation et permet d'en retrouver le contexte des mois plus tard.

```
fix(narration): ne plus perdre la progression sur un marqueur d'etat invalide
...
Fixes: ANO-2026-001
Refs: C4.2.1
```

6.3 Preuve de resolution

La validité est établie en exécutant la **même suite de tests** avant puis après le correctif. Ce double passage est la démonstration recherchée : les tests **échouent effectivement** sur le code d'origine, ce qui prouve qu'ils décrivent l'anomalie et non un comportement quelconque. Les deux tests qui passaient déjà sont ceux du comportement nominal, que le correctif ne devait pas modifier.

Commit	Résultat
b335510 — précédant le correctif	16 failed , 2 passed
26df2a1 — commit du correctif	18 passed

6.4 Validation post-déploiement et retour arrière

Defaut identifié — un déploiement valide sur la mauvaise sonde

Le déploiement se terminait sur un simple `curl de /api/health`, la **sonde de vivacité**, qui répond ok dès le démarrage du processus : une version inapte au trafic était déclarée déployée avec succès. **Aucun retour arrière n'existait.**

Le workflow mémorise désormais le commit en service, exécute `scripts/smoke_test.sh`, et restaure la version précédente en cas d'échec.

```
# Version actuellement en service, conservée pour un retour arriere.
VERSION_PRECEDENTE="$(git -C "$APP_DIR" rev-parse HEAD)"

docker compose -p rpg40k up -d --build
```



```

if ! ./scripts/smoke_test.sh "http://127.0.0.1:$RPG40K_HTTP_PORT"; then
  echo "::error::Test de fumee en echec - retour a $VERSION_PRECEDENTE"
  git -C "$APP_DIR" checkout --force "$VERSION_PRECEDENTE"
  docker compose -p rpg40k up -d --build
  exit 1
fi

```

Le test exerce cinq controles, choisis pour distinguer un service *demarre* d'un service *operationnel*.

#	Controle	Ce qu'il detecte
1	Disponibilite (30 tentatives)	Conteneur non demarre ou build echoue
2	Sonde d'aptitude /api/health/ready	Base injoignable, volume non monte, disque plein
3	Exposition de /api/metrics	Version deployee sans supervision — angle mort
4	/api/state sans jeton renvoie 401	Regression de configuration exposant les routes de jeu
5	Livraison de l'interface	Panne du reverse proxy Nginx

1. Attente de la disponibilite	[OK]	service joignable apres 1 tentative(s)
2. Sonde d'aptitude	[OK]	toutes les dependances sont disponibles
3. Point de collecte des metriques	[OK]	metriques exposees
4. Protection des routes de jeu	[OK]	acces non authentifie refuse (401)
5. Livraison de l'interface	[ECHEC]	interface non servie par le reverse proxy

Le cinquieme controle echoue **volontairement** ici : le script visait le backend seul (port 8000), qui ne sert pas l'interface ; en production la cible est le port du reverse proxy. Ce relevé demontre que le script **detecte reellement une chaine incomplete** plutot que de valider systematiquement.

Garde-fou de securite ajoute au passage : le deploiement creait `.env` par copie de `.env.example`, ce qui signait les jetons de production avec le `JWT_SECRET` d'exemple *public dans le depot*. Le deploiement echoue désormais si le secret est absent, trop court ou laisse a sa valeur d'exemple.

Figure 13 — Executions reelles de la chaine. La CI #86 est verte sur `main` ; Dependabot a ouvert 11 pull requests, dont la montee *majeure* `jest-dom 6 → 7` en PR isolee, conformément a la politique du §2.

Couverture des criteres

Critere	Traitement
Le traitement tire profit de la chaine CI/CD	Branche dediee, commits atomiques referencant l'anomalie, validation par 5 jobs, deploiement conditionne a une CI verte, test de fumee, rollback automatique
Le correctif est decrit et resout l'anomalie	Deux niveaux de correctif code a l'appui ; memes tests avant (16 echecs) et apres (18 succes) ; releve avant/apres sur l'application

7 Recommandations argumentees d'amelioration

Competence C4.3.1 — Proposer des axes d'amelioration en prenant en compte les indicateurs de performance et en analysant les retours utilisateurs afin de maintenir et renforcer l'attractivite du logiciel.

Limite assumee. Le projet n'a pas de base d'utilisateurs constituee : aucun retour reel n'a pu etre analyse. Le canal de collecte (`03_retour_utilisateur.yml`) a donc ete **mis en place** au titre de cette competence, et les recommandations s'appuient sur les indicateurs mesures et les observations de recette. Presenter des retours fictifs aurait prive l'analyse de toute valeur probante.

7.1 Indicateurs mesures

Releve reproductible par `python scripts/benchmark.py` :

```
LATENCE (ms)      p50      p95      POIDS LIVRE AU NAVIGATEUR
/api/health       2.59      3.1      VoxelEngine.js    481.4 Ko    <- 64 % du total
/api/roll         3.2       3.82     index.js          245.7 Ko
/api/state        9.97     11.02    TOTAL JS          756.3 Ko
/api/auth/login  248.56   287.24

BASE : users 87, session_events 141, index : aucun
CONTEXTE LLM  scene 1 : 1209 tokens | scene 10 : 3369 | scene 30 : 8169 (cumul 140692)
```

Lecture. Les routes de jeu (3 a 10 ms) ne sont pas un facteur limitant : aucune optimisation n'est justifiee de ce cote. Les deux points de tension sont le **poids livre au navigateur** et la **croissance du contexte LLM**. La latence du login est **conforme a l'attendu** (219 ms de `bcrypt.checkpw`) et ne doit pas etre reduite.

7.2 R1 — Charger le moteur 3D a la demande

Indicateur	VoxelEngine.js pese 481 Ko sur 756 Ko de JavaScript, soit 64 % du poids livre
Constat	Le jeu est avant tout textuel . Un joueur qui ne declenche jamais de combat 3D telecharge pourtant tout le moteur voxel et Three.js
Cout / delai	Faible — 0,5 a 1 jour ; immediat, sans rupture fonctionnelle
Gain attendu	-64 % de JavaScript au premier chargement ; premier rendu nettement plus rapide, en particulier sur connexion mobile
Risque	Faible : latence au premier combat 3D, absorbable par un indicateur de chargement

Vite produit *deja* des fragments separes (`VoxelEngine-*.js`, `Combat3DDemo-*.js`) : le decoupage existe, seuls les imports restent statiques. Il suffit de les convertir en imports dynamiques (`React.lazy / import()`). Le temps d'affichage initial etant le premier facteur d'abandon d'une application web, c'est le meilleur rapport gain/effort du lot.

7.3 R2 — Borner l'historique envoye au LLM

Indicateur	Contexte de 1 209 tokens (scene 1) a 8 169 (scene 30) ; cumul envoye 140 692 tokens
Constat	<code>session.messages</code> n'est jamais tronque : chaque scene renvoie tout l'historique
Cout / delai	Faible a moyen — 1 a 2 jours ; court terme
Gain attendu	Croissance lineaire au lieu de quadratique ; latence stabilisee ; plafond de contexte jamais atteint
Risque	Moyen — une troncature naive fait « oublier » au maitre du jeu les evenements anciens

Argument de cout, presente honnetement. Le gain financier est faible en valeur absolue : environ **0,021 USD** par partie de 30 scenes contre **0,015 USD** avec une fenetre glissante de 10 messages. Ce n'est **pas** l'argument principal, et le presenter comme tel serait trompeur. Les deux vraies justifications sont ailleurs : la **latence**, qui croit avec le contexte et rompt le rythme de jeu au-dela de quelques dizaines de scenes ; et le **plafond de contexte** du modele, dont le depassement provoquerait une erreur, donc aujourd'hui une bascule en mode degrade.

Mise en oeuvre recommandee : conserver le prompt systeme, les N derniers echanges et un **resume condense** des scenes anterieures, regenere tous les 10 tours — approche qui preserve la continuite narrative, precisement ce que le joueur percoit. La sonde `rpg40k_gm_context_messages` mesurera le gain reel.

7.4 R3 a R5 — Capacite, reproductibilite et securite

#	Axe	Constat et gain	Cout
R3	Expirer les sessions en memoire	Le dictionnaire <code>_sessions</code> n'a ni expiration ni eviction : toute session creee reste residente jusqu'au redemarrage. Une expiration apres quelques heures d'inactivite suffit ; l'etat etant persiste a chaque scene, l'eviction est transparente pour le joueur. Gain : consommation memoire bornee.	0,5 j
R4	Verrouiller les versions Python	Le frontend dispose de <code>package-lock.json</code> et de <code>npm ci</code> ; le backend n'a aucun equivalent . Les bornes hautes du §2 empechent les majeures, mais deux installations a deux dates peuvent encore differer sur les correctifs. Un <code>requirements.lock.txt</code> utilise par le <code>Dockerfile.backend</code> et la CI supprime cette derive.	0,5 j
R5	Limiter le debit sur l'authentification	La supervision detecte une recherche exhaustive mais rien ne la bloque . Le cout de <code>bcrypt</code> (219 ms) freine deja l'attaque, mais constitue aussi un vecteur de deni de service : quelques dizaines de requetes simultanees saturent le processus. Une limitation par IP traite les deux problemes.	0,5 j

7.5 R6 a R8 — Securisation et hygiene

#	Axe	Constat et gain	Cout
R6	Desactiver le mode debug (applique)	<code>FastAPI(..., debug=True)</code> etait ecrit en dur : toute exception non geree exposait la trace d'execution et des extraits de code source en production (OWASP A05). Applique en 1.3.0 : le mode est pilote par <code>API_DEBUG</code> , absente par default, et un test verifie qu'il reste desactive. Le cout etant inferieur a l'heure et l'enjeu relevant de la securite, il n'y avait pas lieu de differer.	< 1 h
R7	Retention sur <code>session_events</code>	141 lignes, aucun index , et surtout aucune lecture dans le code : la table est alimentee par <code>record_event()</code> mais jamais interrogee. Deux options s'excluent : soit la donnee est utile et il faut l'exposer et l'indexer sur <code>user_id</code> , soit elle ne l'est pas et il faut purger. La conserver sans usage cumule cout de stockage et obligation de protection des donnees personnelles, sans contrepartie.	0,5 j
R8	Conteneurs sans privileges root	Ni <code>Dockerfile.backend</code> ni <code>frontend/Dockerfile</code> ne comportent de directive <code>USER</code> . Gain : une execution de code arbitraire n'obtient plus root dans le conteneur. Ajuster les droits sur les volumes <code>/app/data</code> et <code>/app/saves</code> .	0,5 j

7.6 Priorisation et suivi des gains

Rang	Recommandation	Delai	Moyen de verification du gain
–	R6 — desactiver <code>debug=True</code>	Fait (1.3.0)	Controle n°4 du test de fumee
1	R1 — chargement 3D a la demande	Immédiat	<code>benchmark.py</code> , section poids livre
2	R5 — limitation de debit sur le login	Court terme	<code>rpg40k_auth_attempts_total</code>
3	R3 — expiration des sessions	Court terme	<code>rpg40k_active_sessions</code>
4	R2 — fenetre glissante du contexte	Court terme	<code>rpg40k_gm_context_messages</code>
5	R4 — verrouillage des versions Python	Immédiat	Comparaison CI / production
6	R7 — retention <code>session_events</code>	Moyen terme	<code>benchmark.py</code> , section base
7	R8 — conteneurs sans root	Moyen terme	<code>docker compose exec backend id</code>

Charge residuelle : 4 a 5,5 jours-homme, R6 etant deja appliquee. Aucune refonte architecturale : toutes les evolutions sont incrementales, deployables par le processus du §6, et **chacune dispose deja d'un moyen de mesure** (benchmark ou sonde), de sorte que le gain pourra etre constate plutot que suppose.

7.7 Contribution a l'attractivite

Rapidite (R1, R2) : démarrage plus rapide, rythme preserve en partie longue. **Fiabilite** (R3, R4) : service stable, comportement identique d'un environnement a l'autre. **Confiance** (R5, R6, R8) : compte protege, absence de fuite d'information. **Evolutivite** (R7). L'attractivite d'un jeu narratif tient d'abord a la **fluidite de l'experience** : un chargement long ou une attente croissante degradent l'engagement bien avant qu'une fonctionnalite ne manque — d'ou la priorite de R1 et R2 sur toute extension fonctionnelle.

Couverture des criteres

Critere	Traitement
Recommandations argumentees, gains evaluables (cout, delai)	Huit axes, chacun avec indicateur mesure, gain chiffre, cout en jours-homme et priorite
Realistes et realisables	Aucune refonte ; evolutions incrementales deployables par le §6 ; reserves de faisabilite (R4, R8) explicitees
Renforcent l'attractivite	Quatre axes relies a l'effet percu par le joueur, priorisation justifiee de R1 et R2

8 Journal des versions

Competence C4.3.2 — Etablir un journal des versions deployees en y integrant la documentation des correctifs realises pour suivre les evolutions realisees sur le logiciel.

Defaut identifie — le logiciel ignorait sa propre version

`backend/api.py` declarait `version="1.0.0"` en dur alors que le journal en etait a 1.2.0. Or le formulaire de consignation demande au joueur de relever la valeur retournee par `/api/health` : un signalement aurait donc porte une version **erronee**, rendant impossible le rattachement d'une anomalie a un etat du code.

Correctif : fichier `VERSION` a la racine (source unique de verite), `backend/version.py` qui l'expose et lit le journal, et **7 tests** interdisant la derive. La protection a ete verifiee en portant volontairement `VERSION` a une valeur incoherente :

```
$ printf '9.9.9\n' > VERSION && pytest tests/test_version.py -q
FAILED test_la_version_correspond_au_journal
FAILED test_le_journal_documente_la_version_courante      2 failed, 5 passed
```

Toute publication oubliant de documenter sa version fait désormais echouer la CI.

8.1 Exempleaire du journal — version 1.3.0

[1.3.0] — Maintien en condition operationnelle — 19/08/2026

Ref. branche : *fix/ANO-2026-001-marqueurs-etat*.

Corrige

- **ANO-2026-001** — perte de la progression d'une scene sur un marqueur d'etat non numerique. Le flux SSE s'interrompait avant l'evenement `done` et avant `session.save()`. Correctif en defense en profondeur ; 18 tests, 16 en echec avant.
- **ANO-2026-002** — perte *partielle* au redemarrage : `Session.save()` omettait la fiche de personnage. 8 tests, 6 en echec avant.
- **ANO-2026-003** — test d'animation intermittent en CI (flottant de haute precision). 407 cas sur 300 000. Tests rendus deterministes.
- **Version rapportee incorrecte et documentation du deploiement** contredite par le code.

Securite

- Mode debug desactive par default (R6) ; garde-fou `JWT_SECRET` au deploiement ; sonde et alerte sur les tentatives d'authentification.

Ajoute

- *Supervision* : `monitoring.py`, `/api/health/ready`, `/api/metrics`, 18 regles d'alerte, tableau de bord Grafana.
- *Dependances* : `dependabot.yml` (4 ecosystemes), bornes hautes, `check_updates.py`.
- *Deploiement* : `smoke_test.sh` (5 controles), retour arriere automatique.
- *Qualite* : formulaires de consignation, `benchmark.py`, fichier `VERSION`.

Tests — `test_monitoring.py` (20), `test_state_markers.py` (18), `test_version.py` (7), `test_character_persistence.py` (8). **Total : 82 → 135 au vert**, plus 30 tests frontend.

Regles de tenue — une section par version, la plus recente en premier ; rubriques normalisees (*Ajoute*, *Modifie*, *Corrige*, *Securite*, *Tests*) ; chaque correctif nomme l'anomalie, son symptome, sa cause et son correctif ; chaque version reference son tag, ce qui permet le retour arriere ; les rubriques *Securite* sont toujours explicites. Une entree est redigee **au moment du commit**, jamais reconstituee a la publication : c'est la seule facon d'obtenir une description fidele.

La chaine est verifiable **dans les deux sens** : depuis une version affichee par `/api/health` on retrouve l'entree du journal, le commit puis la fiche d'anomalie ; inversement, une fiche mene au commit qui la corrige et a la version qui l'a deployee.

Couverture des criteres

Critere	Traitement
Le journal contient les ameliorations de la version	Exempleaire 1.3.0 : <i>Corrige</i> (4), <i>Securite</i> (3), <i>Ajoute</i> (4 domaines), <i>Tests</i>
Les correctifs deployes sont documentes	Chaque correctif nomme anomalie, symptome, cause racine, correctif et couverture de tests ; correspondance versions / preuves

9 Probleme resolu avec le support client

Competence C4.3.3 — Collaborer avec les equipes de support, en fournissant une expertise technique, en repondant aux retours clients, en resolvant des problemes complexes afin d'ameliorer le logiciel.

Precision de contexte. Le projet est developpe par une personne seule : il n'existe pas d'equipe de support distincte. Les roles decrits correspondent aux **fonctions reellement exercees** — reception et qualification d'une part, expertise technique d'autre part — et non a des personnes distinctes. L'anomalie, son diagnostic et son correctif sont authentiques. Inventer des echanges entre equipes fictives aurait retire toute valeur probante a cet exemple.

9.1 Contexte du retour et probleme a resoudre

Signalement : « Mon personnage est revenu a zero mais j'ai garde mon niveau et mon equipement. » Gravite majeure, perimetre sauvegarde, version 1.2.0, frequence « deux fois, mais pas a chaque fois ». Le joueur est niveau 3 avec tout son equipement, mais son personnage est de nouveau indemne, ses rations pleines, ses points d'attribut disparus.

Trois caracteristiques en font un cas difficile. **1.** La perte est **partielle**, donc contre-intuitive : une perte totale orienterait vers l'ecriture disque, tandis que cette incoherence apparente suggere a tort un probleme d'affichage. **2.** Le symptome parait **intermittent** : la formulation « pas a chaque fois » est exacte mais decrit une correlation que le joueur ne peut pas percevoir. **3.** Le contournement evident est **inoperant** : sauvegarder manuellement ne change rien.

9.2 Contribution des parties prenantes

Niveau 1 — reception. Trois apports decisifs obtenus par le formulaire structure : la **distinction precise** entre donnees perdues et conservees, qui orientera tout le diagnostic ; la **qualification en gravite majeure**, qui declenche une prise en charge sous 48 h meme sans blocage ; et surtout la **reformulation de l'intermittence** — interroge sur ce qui distinguait les sessions touchees, le joueur precise qu'il revenait « le lendemain », par opposition a un simple rechargement. Cette precision transforme un symptome « aleatoire » en condition reproductible : un delai suffisant pour que le serveur ait redemarre. Sans elle, le niveau 2 aurait cherche une anomalie d'ecriture disque, alors que le default se situe au **rechargement**.

Niveau 2 — expertise. L'hypothese est reproduite en vidant les sessions residentes, ce qui simule un redemarrage. Le releve reproduit *exactement* la repartition decrite :

donnee	avant redemarrage	apres	verdict
rations (personnage)	1	3	>>> PERDU <<<
blessures (personnage)	Grave	Indemne	>>> PERDU <<<
points d'attribut	9	remis a 0	>>> PERDU <<<
niveau / xp (progression)	3 / 250	3 / 250	CONSERVE

Cause racine — un sous-systeme absent de la liste

`Session.save()` persiste *six* sous-systemes (progression, inventaire, carte, quetes, relations, equipe) mais omet `self.character`. Au demarrage, la fiche est donc reconstruite depuis le modele vierge. **La liste des donnees perdues correspond exactement au contenu de `CharacterState`, celle des donnees conservees aux six sous-systemes persistes : la repartition observee par le joueur est la signature directe de la cause.**

L'examen confirme qu'il s'agit d'un oubli et non d'un choix : la classe possede `to_dict()` et `save()`, mais **aucune reciproque `from_dict()`**. L'etat pouvait etre serialise mais jamais relu — l'omission n'avait donc jamais pu etre detectee, faute de chemin de lecture. Aucun test ne simulait de redemarrage : le default n'etait observable qu'au rechargement a froid, soit exactement la condition que le joueur avait decrite sans pouvoir la nommer.

9.3 Resolution apportee

Trois volets : ajout de la reciproque `CharacterState.from_dict()` ; persistance de `character.yaml` dans `Session.save()` ; rechargement avec **repli** sur le modele vierge pour les parties anterieures au correctif et en cas de fichier illisible. Cette compatibilite ascendante est une exigence du support : un correctif rendant inutilisables les parties existantes transformerait une perte partielle en **perte totale**.

8 tests de non-regression : **6 en echec** avant correctif, **8 au vert** apres. Les deux qui passaient deja sont ceux de compatibilite ascendante, que le correctif ne devait pas modifier. Un test verifie specifiquement la **coherence** entre fiche et progression, puisque c'est leur divergence qui constituait le symptome.

Reponse au joueur : diagnostic en termes non techniques, explication de l'intermittence, contournement provisoire, version de livraison. Le deuxieme point est le plus important : il **valide l'observation**

du joueur au lieu de la contredire. Un signalement qualifie d'« irreproductible » decourage les suivants, qui sont pourtant la principale source de detection sur un logiciel a faible base d'utilisateurs.

9.4 Amelioration au-dela du correctif

Une **couverture de test manquante** a ete comblee (aucun test ne simulait de redemarrage : c'est desormais le cas pour *tous* les sous-systemes persistes) ; une **asymetrie d'API** a ete corrige (to_dict() sans from_dict()), cause profonde de l'omission ; et le diagnostic a confirme l'interet de la recommandation R3.

Couverture des criteres

Critere	Traitement
Contexte du retour et probleme a resoudre	Fiche telle que recue, puis analyse : perte partielle contre-intuitive, intermittence apparente, contournement inoperant
La resolution apportee	Trois volets, compatibilite ascendante, verification par 8 tests executes avant et apres, reponse formulee au joueur
Contribution des parties prenantes	Apport detaille de chaque fonction, en indiquant ce qui aurait manque sans elle

10 Conclusion

Le dispositif presente ici a ete **construit**, non decrit a partir de l'existant. Le projet disposait au depart d'une chaine d'integration continue et d'un journal des versions, mais d'aucune supervision, d'aucun outillage de veille, d'aucun processus de collecte d'anomalies et d'aucune validation post-deploiement.

Domaine	Etat initial	Etat final
Gestion des dependances	Planchers seuls, majeures subies	Bornes hautes, Dependabot hebdomadaire, veille en CI
Supervision	Aucune ; /api/health n'interrogeait rien	19 sondes, sonde d'aptitude reelle, 18 regles, tableau de bord
Collecte des anomalies	Aucun outil	3 formulaires structures, saisie libre desactivee
Validation du deploiement	curl sur la sonde de vivacite	5 controles fonctionnels, rollback automatique
Indicateurs de performance	Aucune mesure	benchmark.py reproductible
Version rapportee	1.0.0 en dur, journal a 1.2.0	Source unique de verite, coherence testee
Tests backend	82	135

Trois anomalies reelles ont ete decouvertes, reproduites, corrigees et couvertes par des tests qui echouent sur le code anterieur — condition sans laquelle un test de non-regression ne prouve rien. Les deux premieres partagent un trait instructif : dans les deux cas, **une protection existait deja ailleurs dans le meme fichier**. La branche **tracks** interceptait l'exception que la branche **resources** laissait passer ; six sous-systemes etaient persistes quand le septieme etait omis. Le default ne venait pas d'une meconnaissance du probleme, mais d'une **application incomplete d'une solution deja connue** — ce qui plaide pour des tests portant sur la *symetrie* des traitements, et non seulement sur les chemins nominaux.

La charge residuelle des recommandations non appliquees est estimee a **4 a 5,5 jours-homme**, sans refonte architecturale. Les deux axes prioritaires portent tous deux sur la **fluidite percue**, qui conditionne l'engagement davantage que l'ajout de fonctionnalites. Le dispositif rend ces evolutions **mesurables** : chaque recommandation dispose d'un moyen de verification deja operationnel, de sorte que le gain pourra etre constate plutot que suppose.

11 Annexe A — Artefacts ajoutees au depot

Chaque element du dossier correspond a un fichier verifiable. Le tableau ci-dessous recense les artefacts crees ou modifies, avec la competence qu'ils servent.

Fichier	Etat	Role
Gestion des dependances — C4.1.1		
.github/dependabot.yml	cree	Surveillance hebdomadaire des 4 ecosystemes
requirements.txt	modifie	Bornes hautes interdisant les montees majeures
scripts/check_updates.py	cree	Rapport de veille classant les obsolescences
.github/workflows/ci.yml	modifie	Veille integree au job security-audit
Supervision — C4.1.2		
backend/monitoring.py	cree	Sondes, middleware, journalisation, sonde d'aptitude
backend/api.py	modifie	/api/health/ready, /api/metrics, instrumentation
monitoring/prometheus.yml	cree	Collecte, cibles, auto-supervision
monitoring/alert_rules.yml	cree	18 regles reparties en 6 groupes
monitoring/alertmanager.yml	cree	Routage par severite, groupement, inhibition
monitoring/grafana/	cree	Tableau de bord provisionne (13 panneaux)
docker-compose.monitoring.yml	cree	Pile de supervision en surcouche
Anomalies et support — C4.2.1, C4.3.3		
.github/ISSUE_TEMPLATE/	cree	3 formulaires, saisie libre desactivee
src/state.py	modifie	Correctifs AN0-2026-001 et AN0-2026-002
Deploiement — C4.2.2		
scripts/smoke_test.sh	cree	5 controles post-deploiement
.github/workflows/deploy-vps.yml	modifie	Test de fumee, rollback, garde-fou JWT_SECRET
Performance et versions — C4.3.1, C4.3.2		
scripts/benchmark.py	cree	Mesure des indicateurs de performance
VERSION	cree	Source unique de verite du numero de version
backend/version.py	cree	Lecture de la version et du journal
CHANGELOG.md	modifie	Entree [1.3.0] documentant les correctifs
frontend/scripts/capture_screenshots.mjs	cree	Captures d'ecran reproductibles de ce dossier

12 Annexe B — Inventaire des tests

La suite backend passe de **82 a 135 tests**. Les quatre fichiers ajoutes couvrent exactement les dispositifs et les anomalies decrits dans ce dossier.

Fichier de test	Tests	Avant correctif	Ce qu'il garantit
tests/test_monitoring.py	20	—	Normalisation des libelles (cardinalite bornee), comptage du trafic par famille de code, sondes de securite, et surtout : la sonde d'aptitude detecte reellement une base corrompue et un fichier de jeu absent.
tests/test_state_markers.py	18	16 en echec	Aucune valeur textuelle ne leve d'exception ; un marqueur rejete ne corrompt pas l'etat ; un marqueur invalide n'annule pas ses voisins valides ; le flux SSE emet done et session.save() est atteint.
tests/test_version.py	7	—	Coherece entre VERSION , la valeur exposee par /api/health et l'entree la plus recente du journal ; mode debug desactive par default.
tests/test_character_persistence.py	8	6 en echec	Ressources, blessures et points d'attribut survivent a un redemarrage ; coherence fiche / progression ; compatibilite ascendante avec les sauvegardes anterieures et repli sur fichier illisible.
Total ajoute	53	22 en echec	

La colonne *Avant correctif* est la plus importante : elle atteste que les tests de non-regression **echouent effectivement** sur le code anterieur. Un test qui passerait dans les deux etats ne prouverait rien de l'anomalie qu'il pretend couvrir.

Cote frontend, les **30 tests** existants sont conserves ; cinq d'entre eux ont ete rendus **deterministes** au titre de AN0-2026-003, sans modification du code de production.